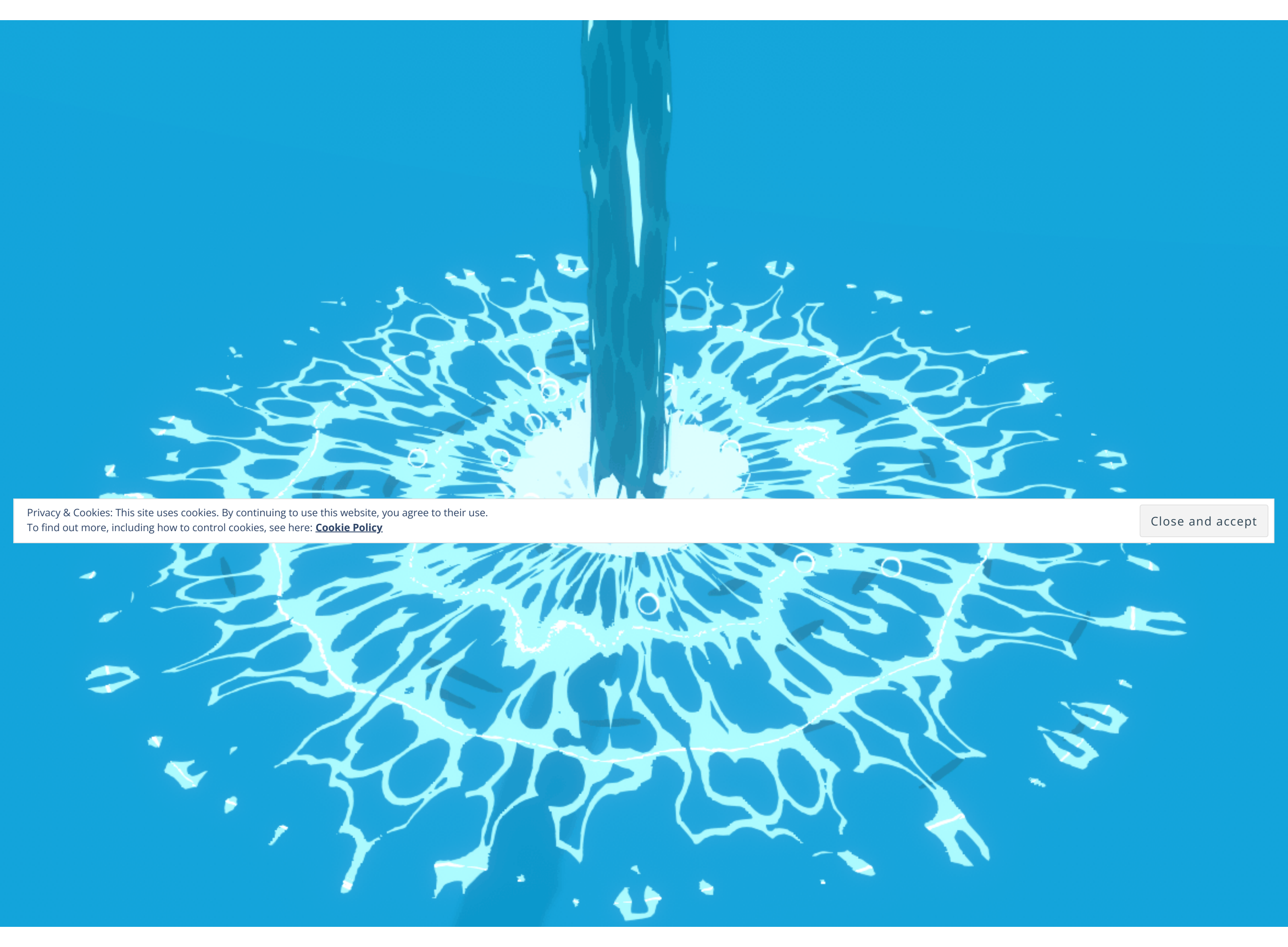


Privacy & Cookies: This site uses cookies. By continuing to use this website, you agree to their use.
To find out more, including how to control cookies, see here: Cookie Policy

Close and accept

How to make Kristof Dedene's anime/ghibli inspired water shader in Unity 3D.

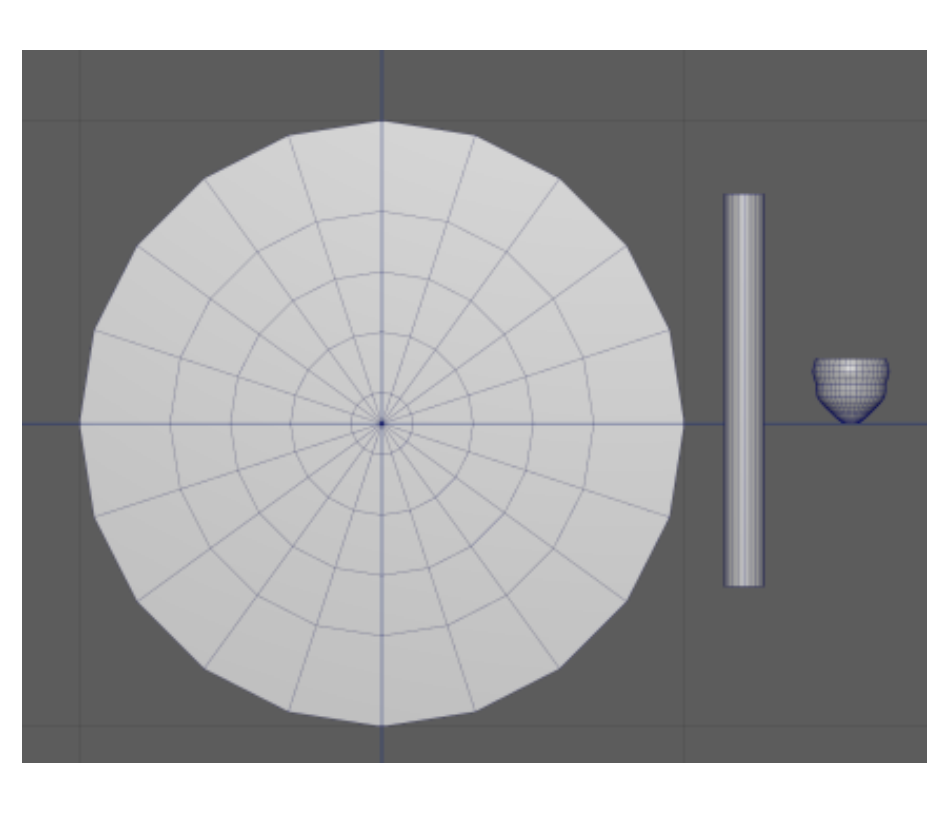
january 14, 2021 by euclid, posted in art

Today I saw **Kristof Dedene's tutorial titled "Tutorial: Advanced anime water splash FX in blender"** and I thought it would be cool to try and attempt to make it for real time use in Unity using URP and Shadergraph, instead of Blender.

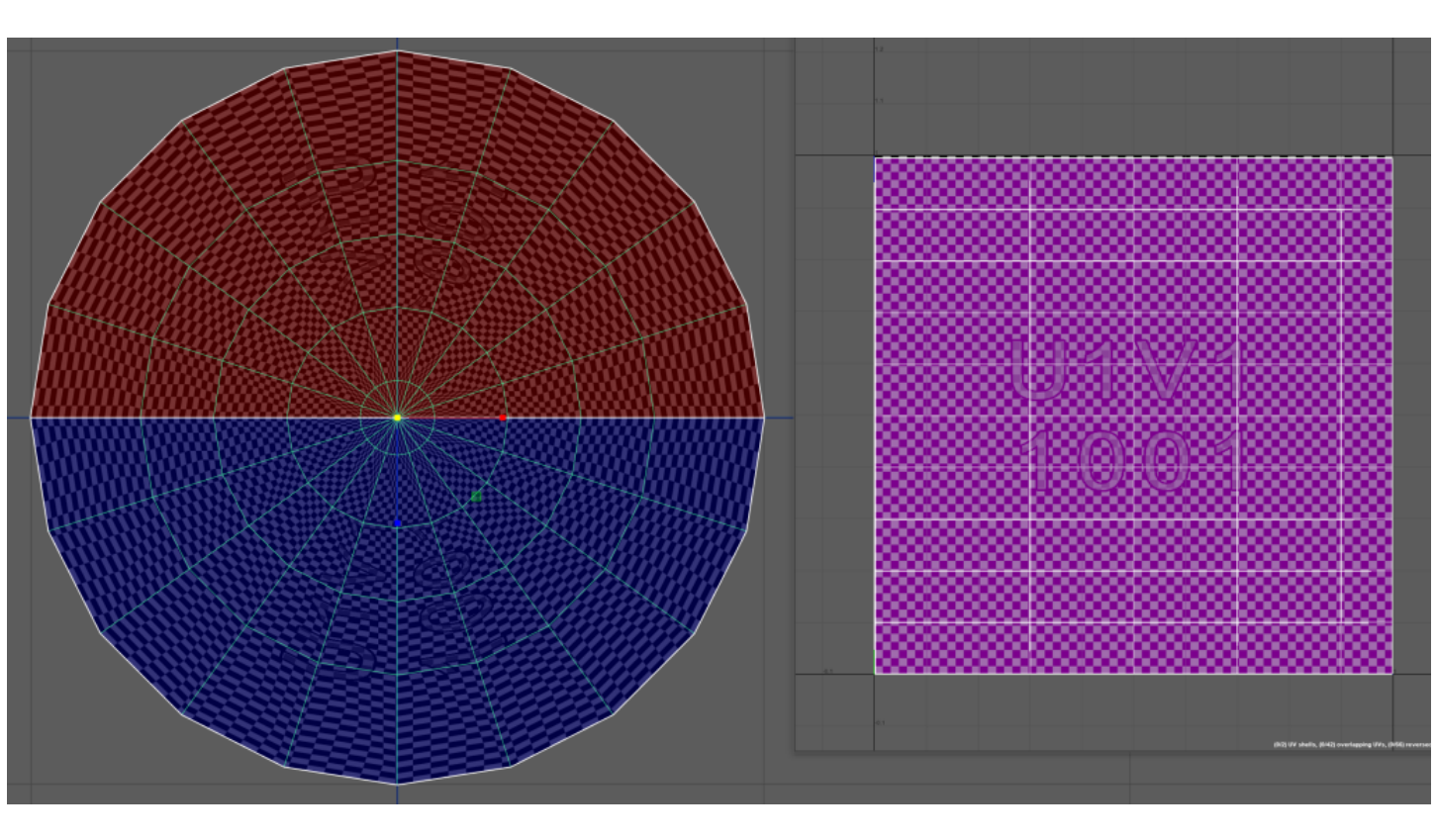
So, where do we begin? Well if you haven't seen his tutorial first, I would highly recommend watching it first, then come back here and see how you could port it.

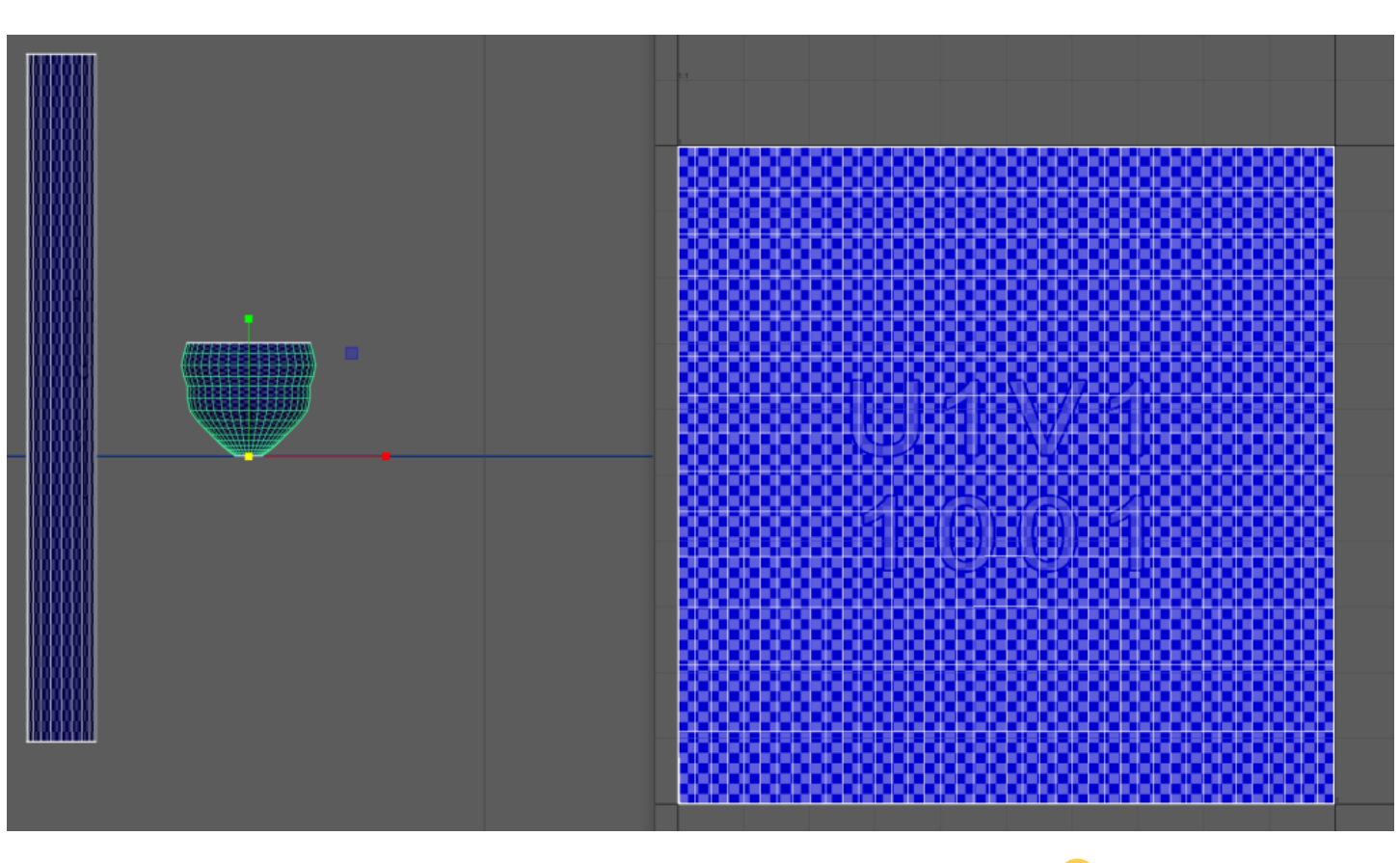
Firstly I noticed that there are a lots of nodes he uses that are straight up incompatible with real time applications so this is not a 1:1 copy of his nodes, that would be impossible, and inefficient for real time.

His water is composed of 3 elements. A deceiving disk that is uv'd straight (I'll get to that later), a cylinder that is used as the spout of water, and the splash mesh that is shaped like a cone.



I'll go into a bit more detail than he did on how to create these meshes because he is using a bunch of modifiers that might not work the same way in other applications. With that being said, the tricky mesh we have here is the ripple disk. You can create this the same way he did if you have blender, and if you have maya you can use the bend modifier much the same way, but when it comes to the UV's you will need to uv it straight like you would a pipe, and normalize it so it takes up the entire UDIM. It is also mirrored (although this really isn't necessary, you just get a bit more resolution, at the cost of a slightly noticeable seam line).



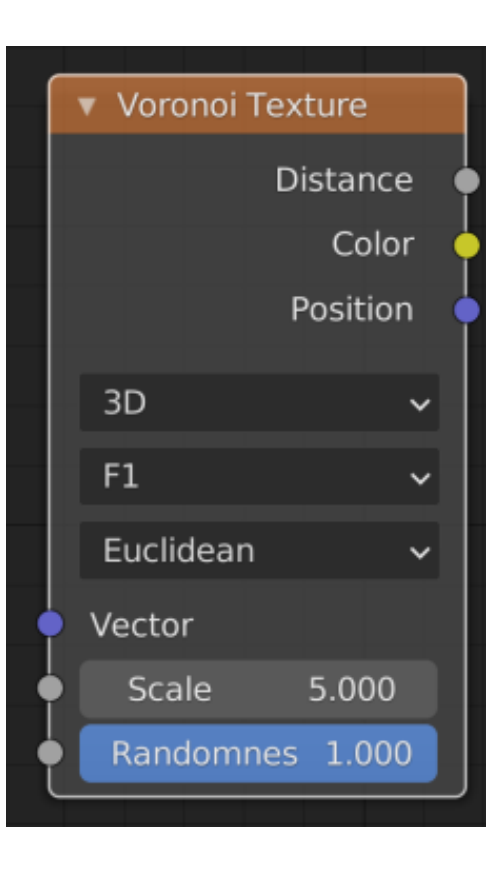


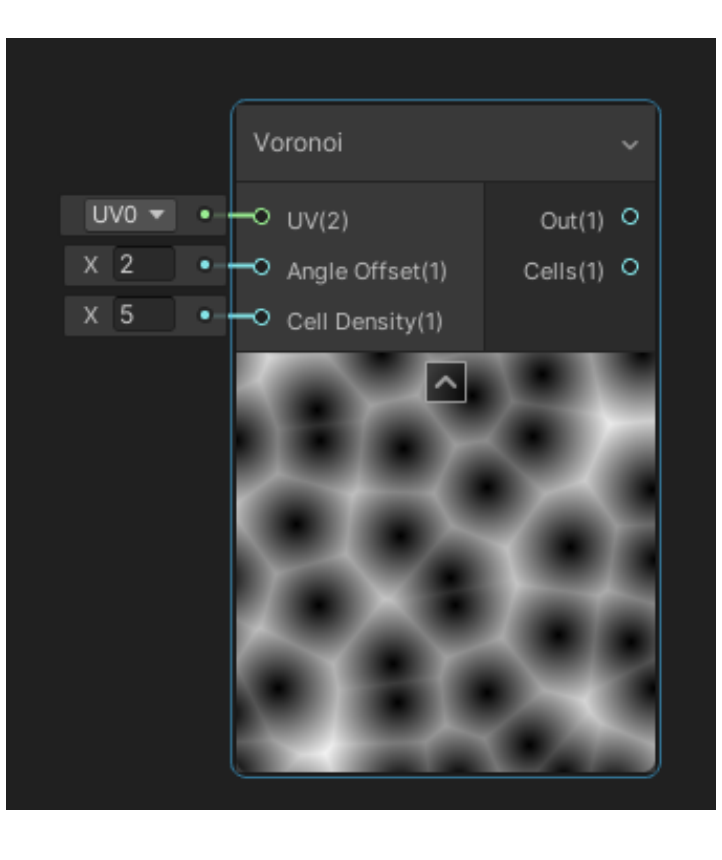
Unwrap + straighten + normalize these as well 🙂

Give each of these bad boys their own material and it might be a good idea to export all the meshes as separate files so you can customize it easier later.

Okay so now we have the problem of what the heck we need to do in Unity. Here are some of the compatibility issues with the how the system in Unity works.

Problem A:
Voronoi texture node in Blender "kind of" exists in Unity as well

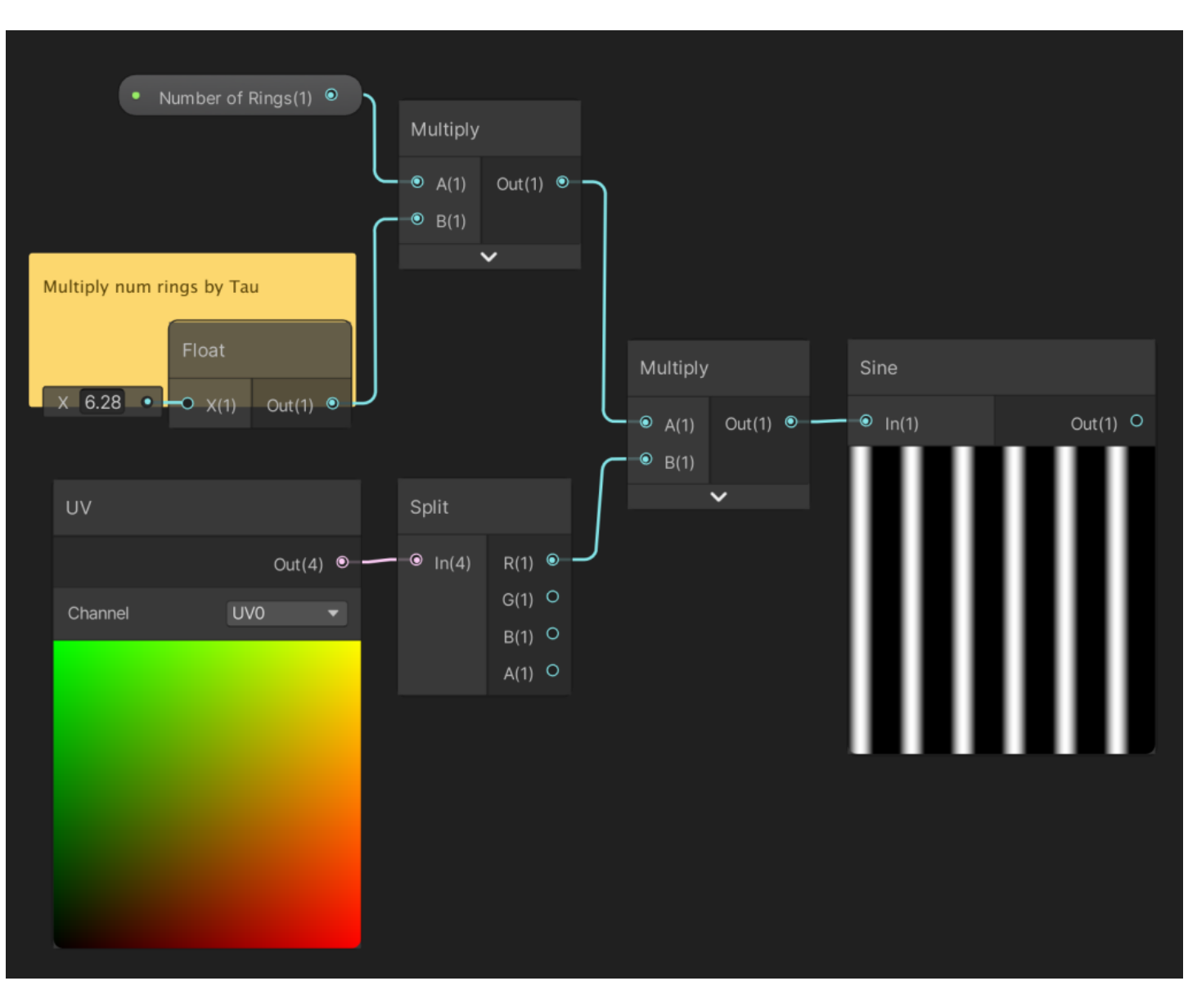




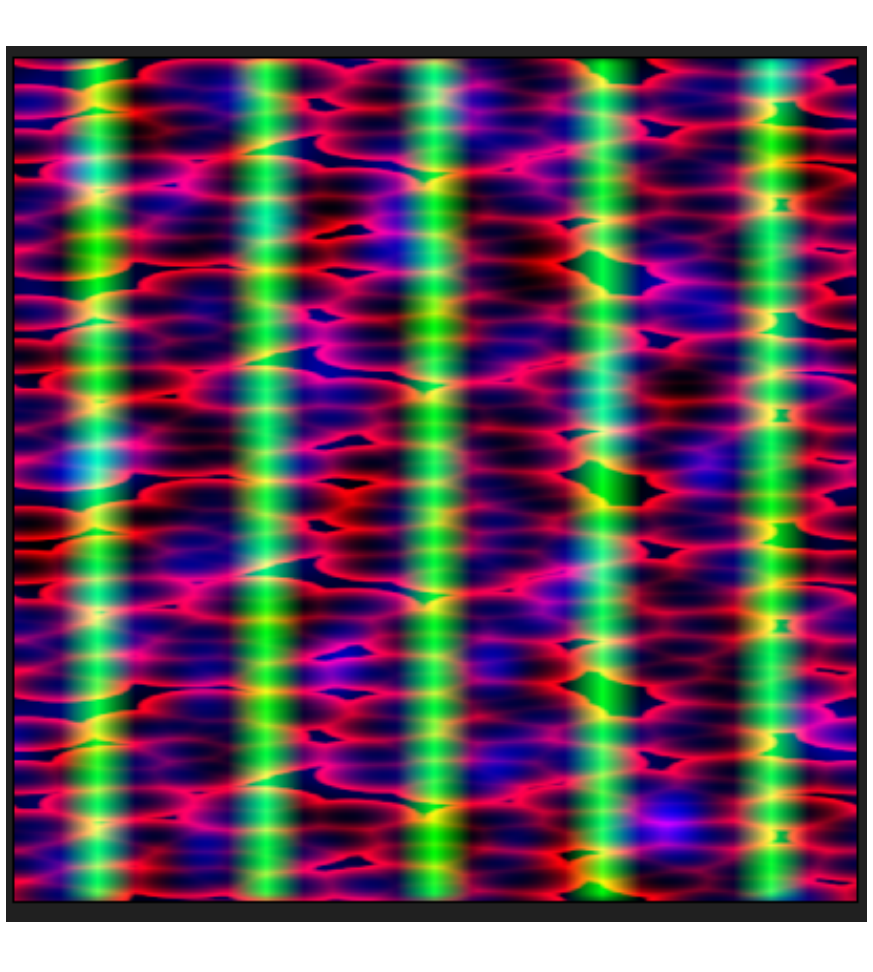
The biggest difference is how he gets the result when it duplicates the voronoi and uses smooth f1 and subtracts it from the previous voronoi texture. The issue with doing the same in unity is A, there is no "smooth voronoi" in Unity. You could use smooth step, but we are adding a bunch of useless nodes now and it's going to get expensive fast. He also uses clouds noise and multiple other instances of voronoi.

Problem B:
The ripples can just be gradient pillars in Unity, but generated in the way that he did in Blender, since the uv's are layed out where a flat gradient will be a radial gradient in 3D space.

The way he generates the ripples in blender is not going to be the same in Unity at all, but there is an option to generate something similar in Shader Graph mathematically with something like this.

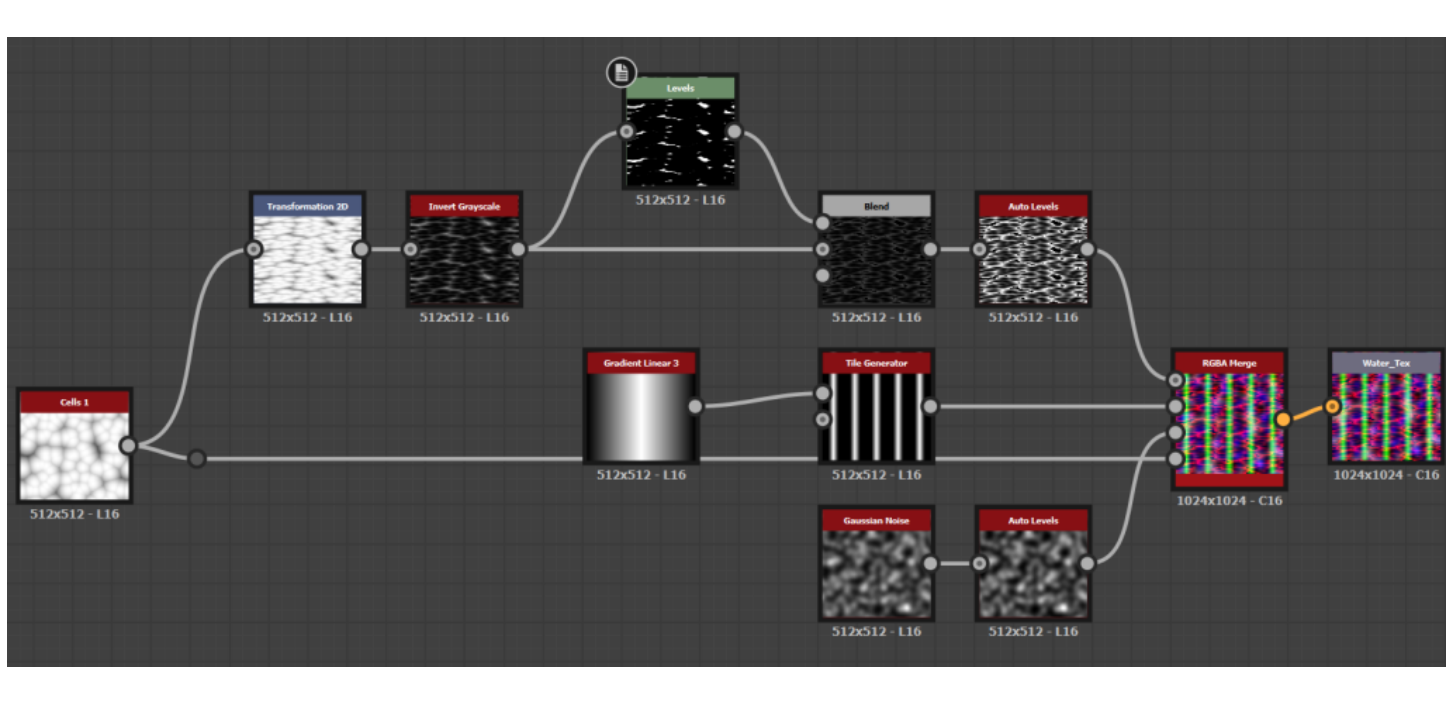


But the issue here is we want to save computation because it's real time, so there seems to be one solution that lets us get there, and it is to generate the textures before and pack them into a RGBA texture.

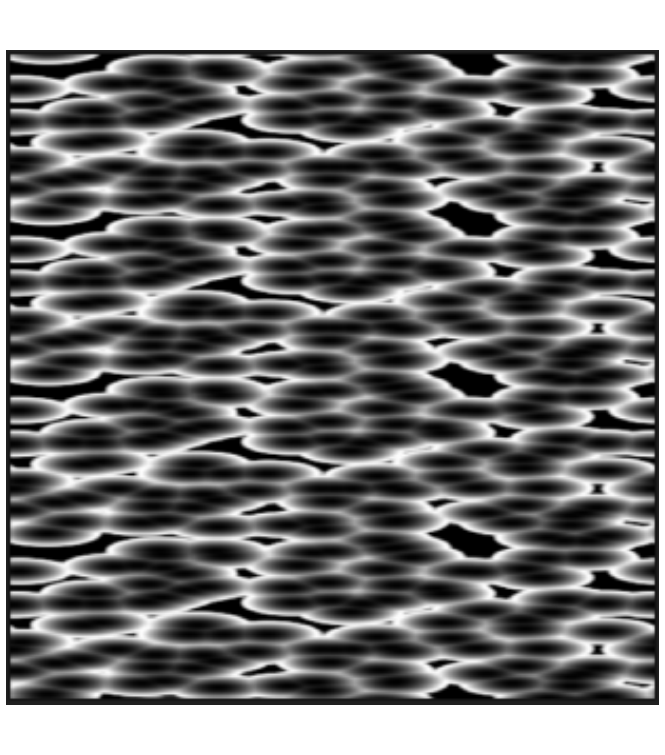


Solution for A and B:
Substance Designer (or equivalent) will suffice for pre generating these tileable textures and we can fit everything we need into one 1024 x 1024 texture.

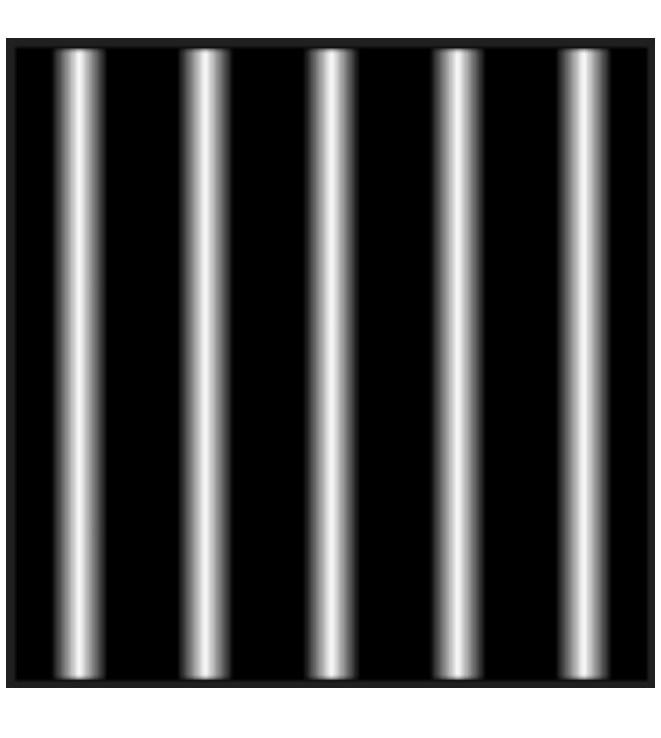
This is how I have my substance designer file setup for this step.



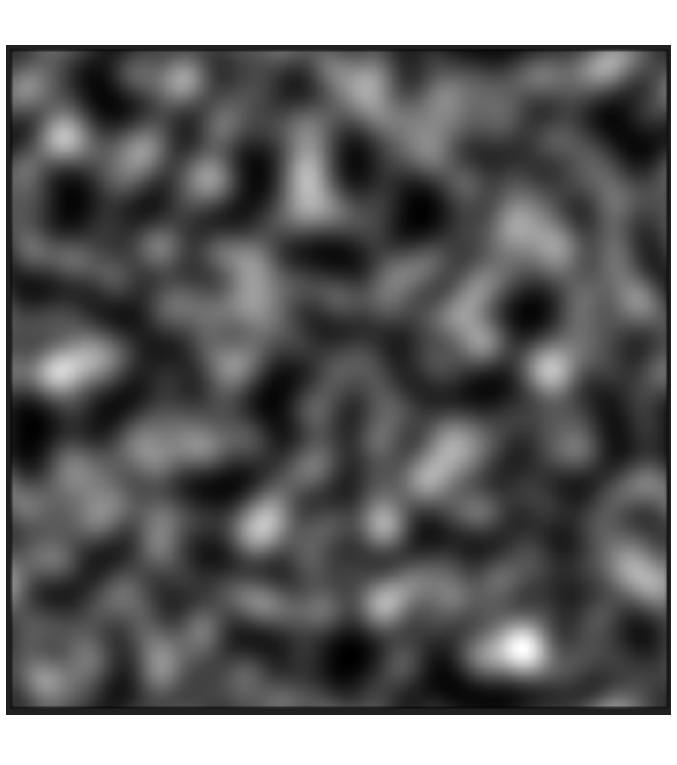
For the R channel we have the result of the 2 voronoi subtracting which looks like this.



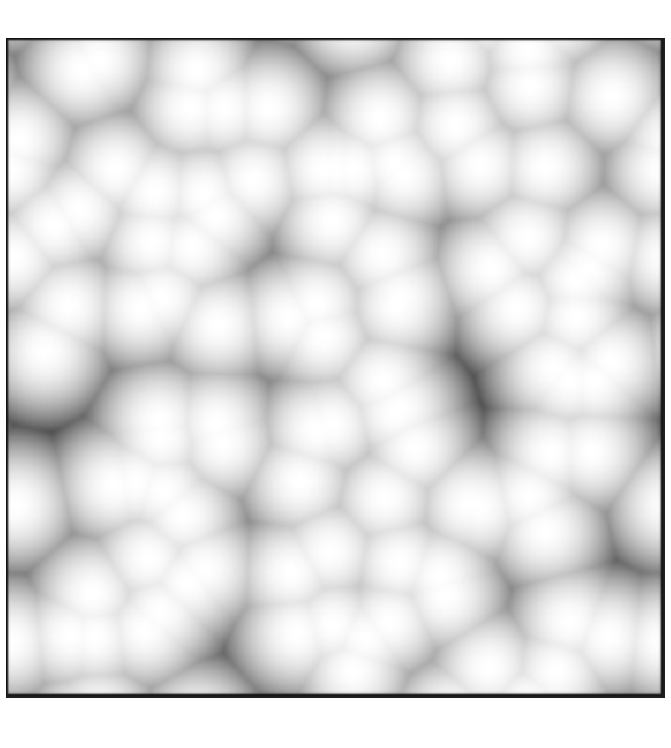
For the G channel I put in the pre tiled pillars that we will be panning to create the ripple effect. I'm just running a Gradient Linear node into a tile generator.



For the B texture I just have a Gaussian Noise.

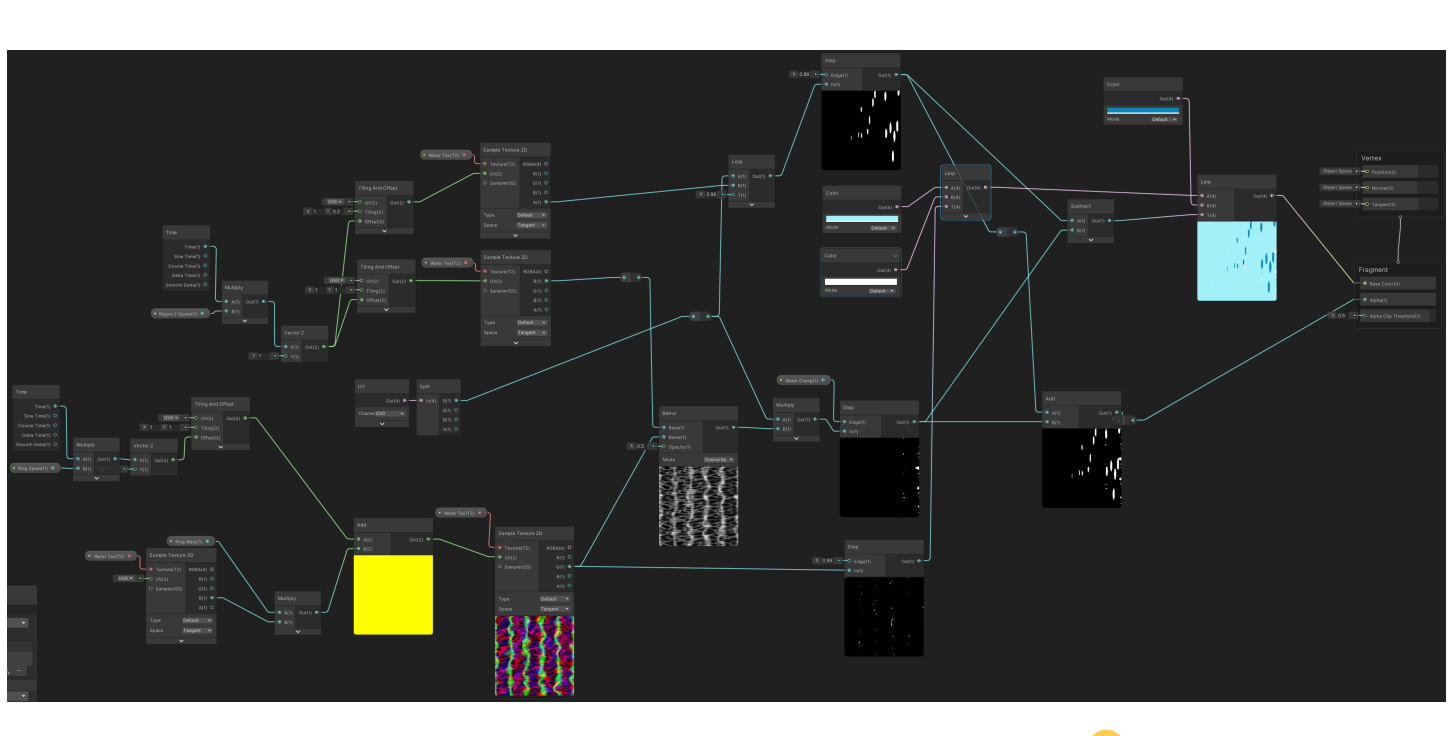


For the A channel I have the original cells 1 texture (voronoi equivalent).



Okay now let's go to Unity.

The Ripple Shader:



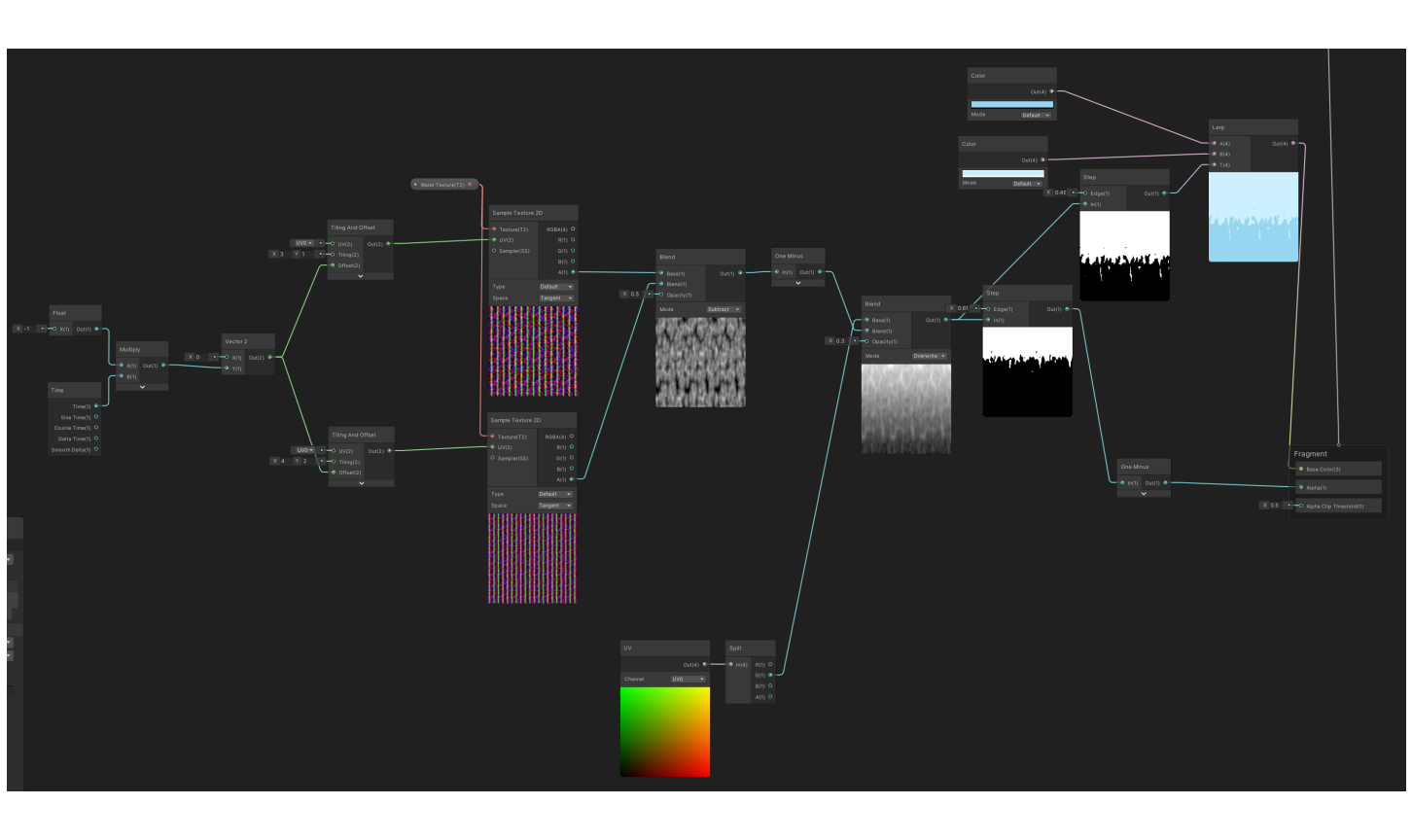
right click image open new tab to see full size 🙂

Everything is sort of the same except I combined the darker ripples, and am just sampling the texture 3 times due to different speeds that it needs to be offset by, but it's still cheaper than using the Voronoi node multiple times in Shader Graph.

Some notable areas include **Step** instead of using the **Gradient Sampler** in shader graph. Step is way cheaper than the gradient sampler. Using **Blend** instead of **Mix** when combining, and using **Lerp** instead of **Blend** to blend based off of an alpha.

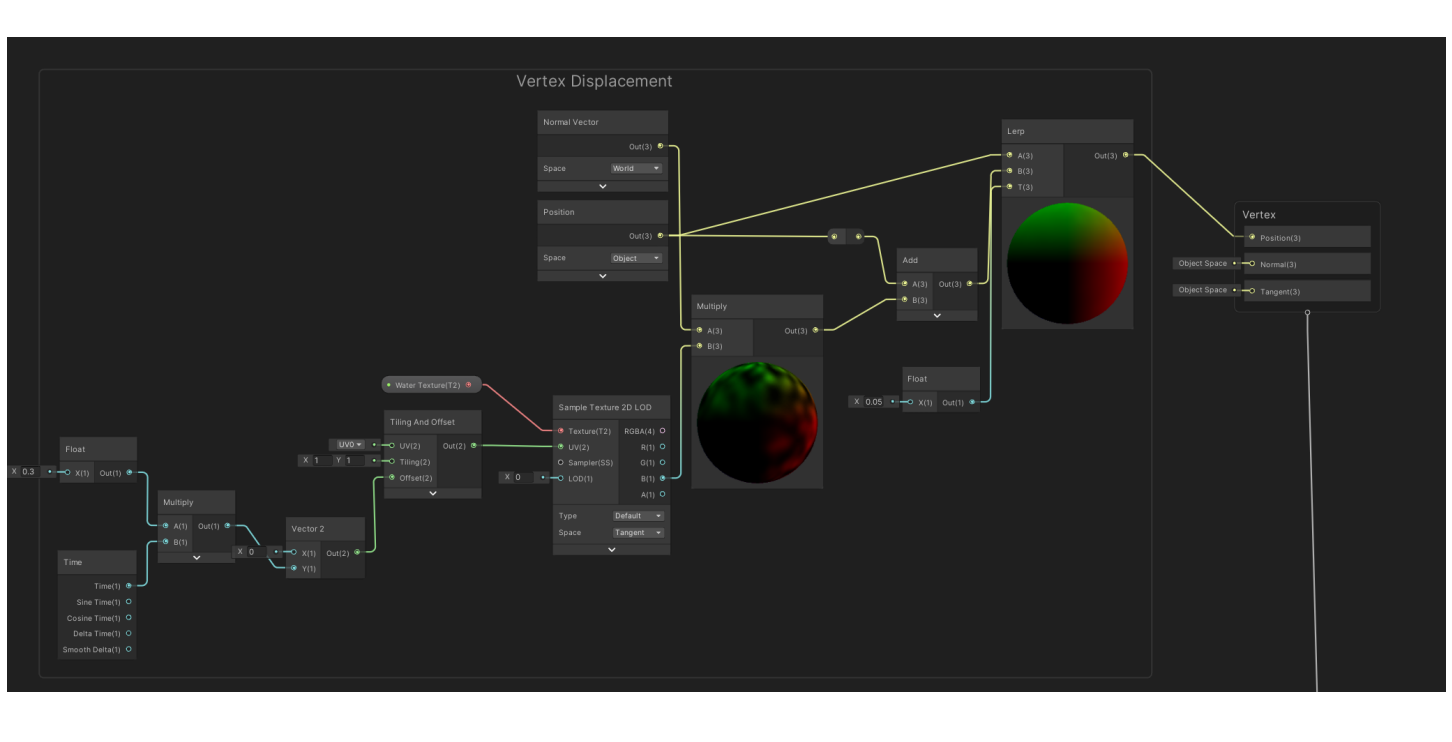
The graph should be pretty self explanatory for the average shader graph user, but **I will be providing the source files that you can take apart and play around** 🙂

The Splash Shader:

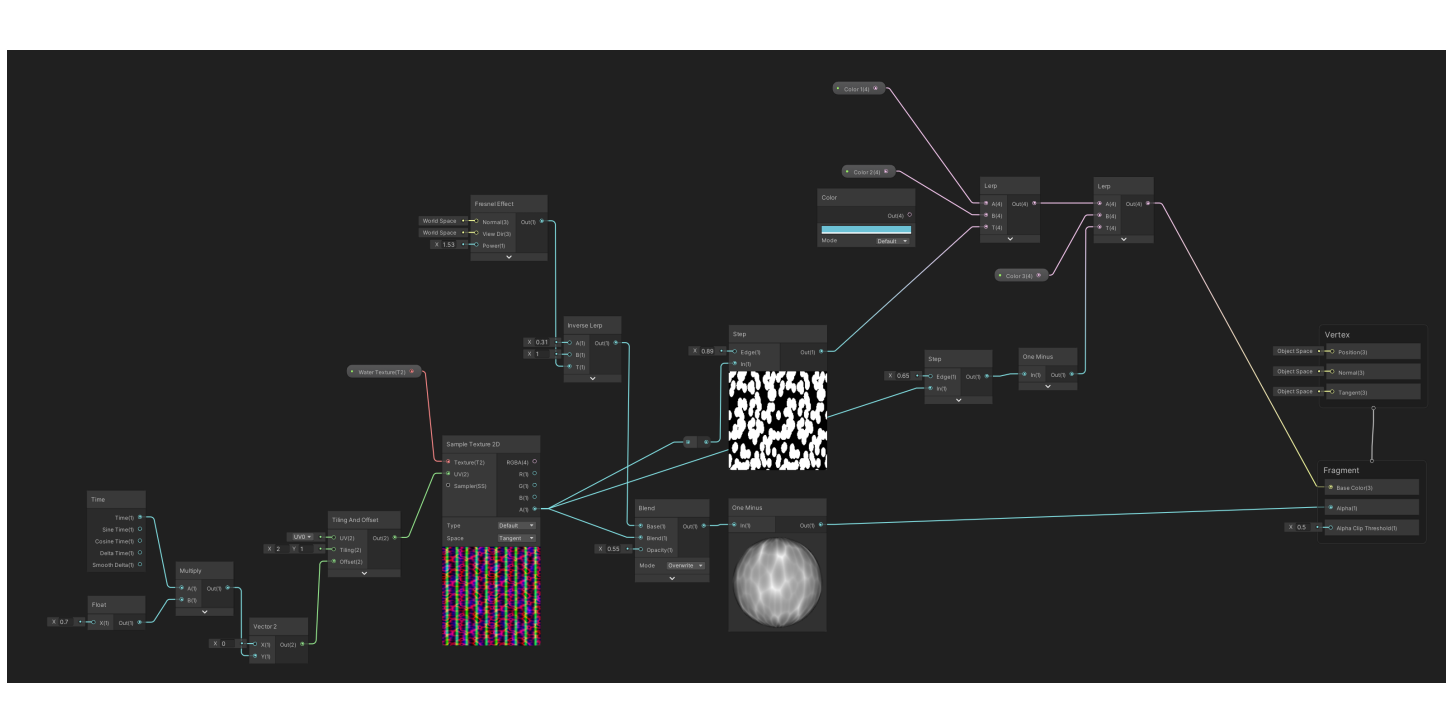


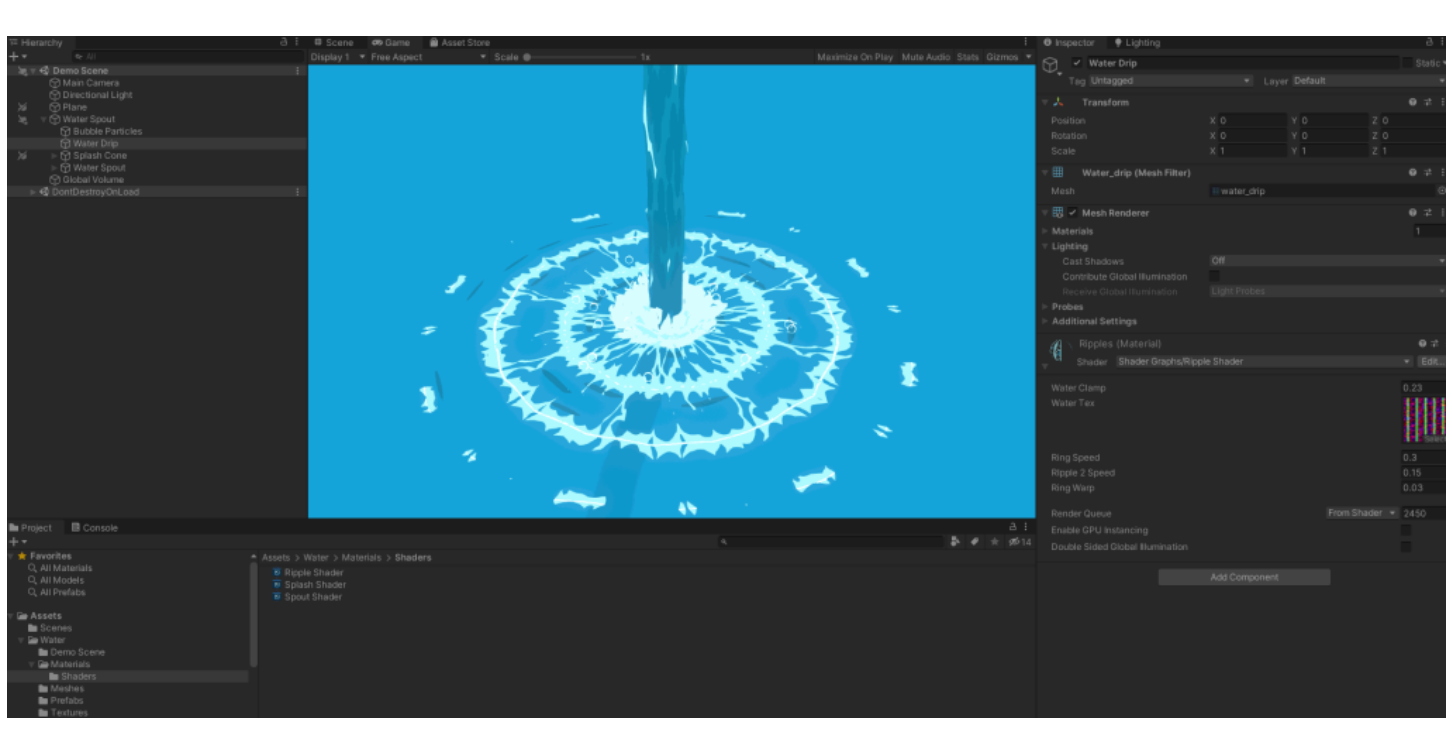
I want to emphasize that I'm using the voronoi in the A channel to basically replace any time he uses a voronoi node.

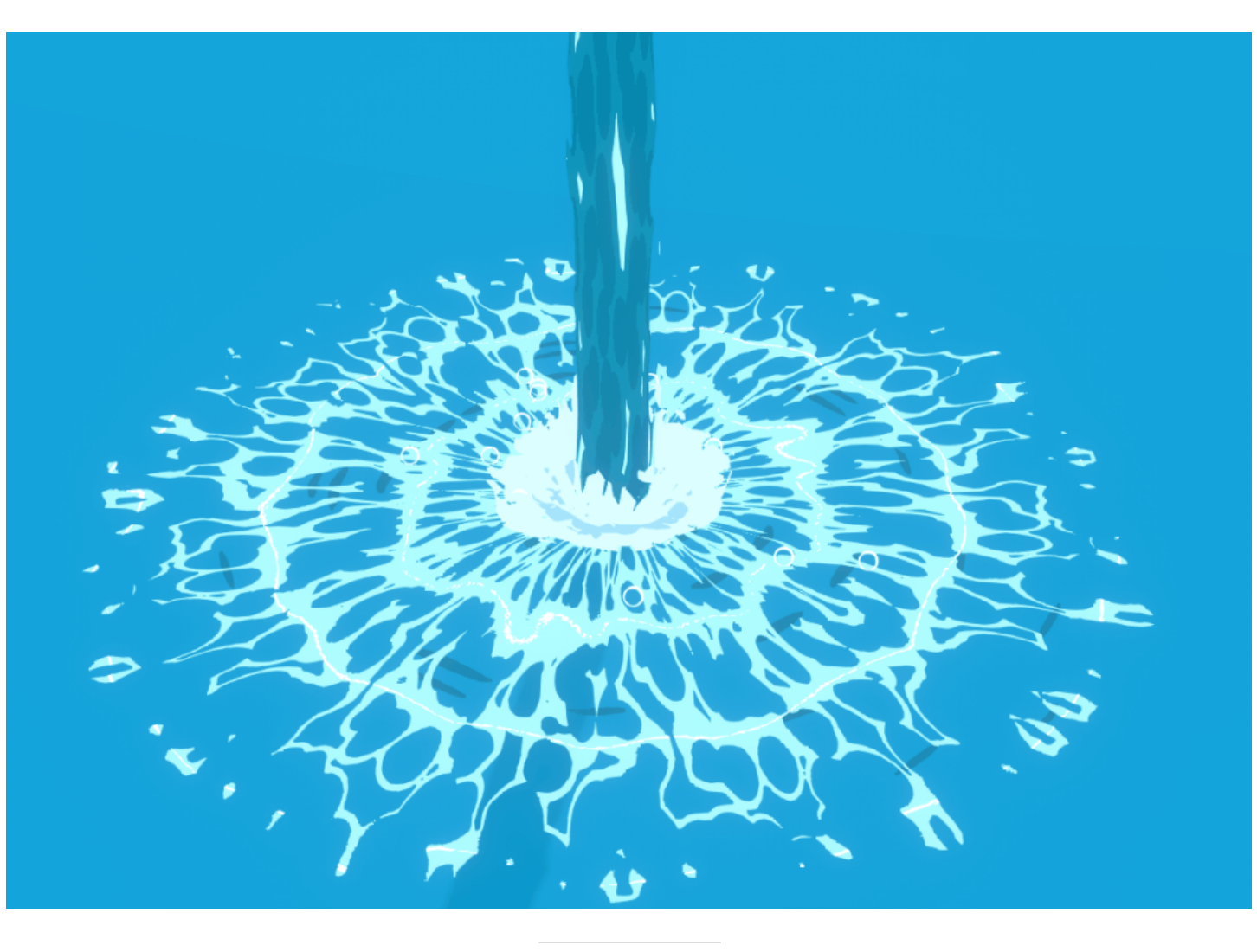
The height displacement part can be plugged in here as well.



The Spout Shader:







Anyway, to wrap up here is the Unity files you can play around with. Have fun!

DOWNLOAD SOURCE FILES

SHARE THIS:

Twitter Facebook

tagged blender, kristof dedene, shader, shader graph, unity, urp, youtube

PREVIOUS POST
New domain!

LEAVE A REPLY

Enter your comment here...